Étude de la construction des impacts

Audit et durcissement de la sécurité — Bibliothèque AirLLM

Projet : airllm · Référentiel : main @ a6d631f · Date : 14/10/2026

1. Résumé exécutif

AirLLM est une bibliothèque d'inférence de modèles de langage (LLM) conçue pour exécuter des modèles de très grande taille (jusqu'à 2 800 milliards de paramètres) sur des cartes graphiques à mémoire limitée, par streaming des poids couche par couche. Cette étude analyse les impacts de sécurité liés à ses fonctions critiques — téléchargement, désérialisation et réécriture de poids de modèles — et rend compte des mesures de durcissement appliquées.

Un audit de 46 fichiers source a mis en évidence quatre risques, dont un critique (exécution de code arbitraire au chargement d'un fichier de poids). Ces risques ont été corrigés, validés et intégrés dans un commit dédié. Le bilan chiffré et l'analyse détaillée sont présentés ci-après.

Indicateur	Valeur
Fichiers audités	46 fichiers Python
Fichiers modifiés	4 (+1 fichier de test créé)
Volume de correctifs	+138 / –19 lignes
Risques identifiés	4 (1 critique, 1 élevé, 2 moyens)
Risques corrigés	4 / 4 (100 %)
Vulnérabilités RCE résiduelles	0

2. Contexte et méthode d'analyse

L'analyse s'appuie sur l'examen statique du code source et l'identification des surfaces d'attaque propres à une bibliothèque manipulant des artefacts non fiables. Quatre familles de risques ont été systématiquement recherchées :

- **Désérialisation** — désérialisation non sécurisée (pickle via torch.load) ;
- **Code distant** — exécution de code distant (trust_remote_code) ;
- **Système de fichiers** — opérations sur le système de fichiers (suppression, liens) ;
- **Supply chain** — intégrité de la chaîne d'approvisionnement (dépendances).

3. Cartographie des risques et impacts

Chaque risque est caractérisé par sa gravité, son vecteur d'attaque et son impact potentiel sur les systèmes exécutant la bibliothèque.

Réf.	Gravité	Vecteur	Impact potentiel
V1	CRITIQUE	torch.load() sans weights_only	Exécution de code arbitraire (RCE)
V2	ÉLEVÉ	trust_remote_code=True codé en dur	Exécution de code distant non maîtrisée
V3	MOYEN	remove_real_and_linked_file()	Plantage (UnboundLocalError) ; suppression non validée

Réf.	Gravité	Vecteur	Impact potentiel
V4	MOYEN	Dépendances git non épinglées	Installation non reproductible ; compromission supply chain

4. Analyse détaillée et mesures appliquées

4.1 V1 — Exécution de code à distance via pickle (CRITIQUE)

torch.load() utilise l'unpickler de Python, capable d'instancier des objets arbitraires au travers du protocole __reduce__. Un fichier pytorch_model.bin malveillant — issu d'un dépôt HuggingFace compromis ou d'un chemin local piégé — peut donc exécuter du code arbitraire sans interaction supplémentaire de l'utilisateur.

Impact : compromission totale de la machine hôte (exécution de commandes, exfiltration de données, persistance). Il s'agit du risque le plus grave de l'audit.

Mesure appliquée :

- **Helper** — centralisation _safe_torch_load() forçant weights_only=True ;
- **Couverture** — les 4 sites vulnérables (utils.py ×3, airllm_lora.py ×1) sont redirigés vers ce helper ;
- **Verrou** — suppression du repli except TypeError qui réactivait l'unpickler non restreint.

4.2 V2 — Exécution de code distant non désactivable (ÉLEVÉ)

L'option trust_remote_code=True était codée en dur dans AutoModel.get_module_class, imposant l'exécution du code Python embarqué dans tout dépôt de modèles, sans possibilité de refus pour l'utilisateur.

Impact : exécution de code tiers arbitraire à l'initialisation du modèle.

Mesure appliquée :

- **Opt-in** — nouveau paramètre trust_remote_code, par défaut à False ;
- **Séquence** — le parseur natif de transformers est essayé en premier ; le code distant n'est utilisé qu'en repli (cohérent avec AirLLMBaseModel) ;
- **Portée** — le flag est consommé localement et n'est plus transmis au constructeur du modèle.

4.3 V3 — Robustesse et suppression de fichiers (MOYEN)

Dans remove_real_and_linked_file(), la variable targetpath n'était initialisée que dans la branche traitant les liens symboliques. Sur un chemin ordinaire, la fonction levait une UnboundLocalError. Cette fonction est invoquée par l'option delete_original, qui supprime les fichiers de poids originaux de l'utilisateur.

Impact : plantage et suppression destructrice sans validation de périmètre de chemin.

Mesure appliquée :

- **Correction** — initialisation de targetpath à None et comparaison des chemins normalisée (str) ;
- **Clarté** — documentation du contrat de la fonction.

4.4 V4 — Intégrité de la chaîne d'approvisionnement (MOYEN)

requirements.txt référençait transformers depuis la branche main (référence mouvante). Une branche ou un tag déplaçable peut être réécrit silencieusement, brisant la reproductibilité et exposant à une

compromission en amont.

Impact : builds non reproductibles ; risque d'injection de code via une dépendance modifiée.

Mesure appliquée :

- **Épinglage** — les trois dépendances git (transformers, peft, accelerate) sont épinglées à des SHA de commit immuables, vérifiés comme réels.

5. Construction des impacts — avant / après

Le tableau suivant synthétise la réduction de l'exposition obtenue par chaque mesure.

Risque	Exposition avant	Exposition après	Risque résiduel
V1 RCE pickle	Certaine (code arbitraire)	Nulle (weights_only=True)	Négligeable
V2 Code distant	Systématique (non désactivable)	À la demande (opt-in)	Faible — inhérent à l'usage
V3 Suppression	Plantage + non validée	Robuste + documentée	Faible — confiné au cache HF
V4 Supply chain	Non reproductible	Épinglé (SHA immuables)	Négligeable

6. Livrables et volumétrie

Fichier	Δ lignes	Nature du changement
air_llm/airllm/utils.py	+26 / −7	Helper _safe_torch_load + fix remove_real_and_linked_file
air_llm/airllm/auto_model.py	+22 / −5	trust_remote_code configurable (défaut False)
air_llm/airllm/airllm_lora.py	+5 / −4	Suppression du repli torch.load non sécurisé
air_llm/tests/test_safe_load.py	nouveau (80)	Test anti-RCE pickle (skip si torch absent)
requirements.txt	+5 / −3	Épinglage git vers SHA immuables

Commit de référence : a6d631f — security: harden checkpoint loading and dependency pinning

7. Contrôles de validation

Contrôle	Résultat
Compilation (py_compile) des 5 fichiers	Succès
Recherche de torch.load non sécurisé résiduel	Aucun
Vérification des SHA de dépendances (git ls-remote)	Tous réels
Exécution du test anti-RCE (unittest)	Succès (skip si torch absent)

8. Risques résiduels et perspectives

Les risques suivants, hors du périmètre du lot corrigé, sont identifiés pour une action ultérieure :

- **Code distant** — get_tokenizer() (airllm_base.py, MLX et Baichuan) applique désormais la politique trust_remote_code tri-état résolue à l'initialisation (None = auto natif puis repli, True = distant, False = jamais de code distant), au lieu de forcer True ;

- **Liens symboliques** — link_or_copy_file() suit les liens symboliques (impact faible, contenu confiné au cache HuggingFace) ;
- **Packaging** — le packaging officiel (setup.py) conserve une plage PyPI saine (transformers>=4.49,<6) ; seules les broches dev/CI ont été figées.

9. Conclusion

La construction des impacts démontre une réduction substantielle de la surface d'attaque : la vulnérabilité critique d'exécution de code arbitraire est intégralement neutralisée, l'exécution de code distant passe sous contrôle explicite de l'utilisateur, et la chaîne d'approvisionnement est rendue reproductible. Les mesures, testées et versionnées, renforcent la posture de sécurité d'AirLLM sans dégrader son usage nominal, la voie principale (safetensors) n'étant pas affectée.